

Lab 3: Campus Cafe Order System

Session 3 (Part 1): Classes, Constructors, Getters & Setters

Duration	90 minutes
Format	Individual work, pure Dart. Every student has personalised numbers.
Slides	Classes & Objects Basics through Getters and Setters
Tools	DartPad (dartpad.dev) or the Dart SDK (<code>dart run --enable-asserts lab3.dart</code>)
Submission	10 checkpoints: your code and your printed output for each step
Total marks	100

Learning Objectives

By the end of this lab you will be able to:

- Declare a class with fields and create independent objects from it
- Write constructors with the `this.` shorthand, including extra logic in the constructor body
- Add named constructors and explain how they differ from the main constructor
- Use a factory constructor to return one shared (cached) instance
- Use an initializer list to compute `final` fields, and `assert` to catch programmer mistakes
- Expose computed data with a getter and protect data with a validating setter

Your Personal Version of This Lab

Every student gets **different numbers**, so every student's printed answers are different. The starter code reads the **last two digits of your roll number** and turns them into a seed:

- `seed` is the two-digit number (00 to 99). For roll number `21F-0347` the seed is 47.
- `t` is the tens digit (4) and `u` is the units digit (7).

Every price, quantity, limit and discount in the lab is calculated from `t` and `u`. If a classmate has different last digits, their output will not match yours, and that is intended. Two students whose roll numbers end in the same two digits will get the same results, but must still submit their own code.

Rules: do not edit the seeded settings in the starter file; never use `t` or `u` as names of your own local variables; use whole-number (`int`) arithmetic only, and use `~/` for division (plain `/` produces a `double` and will not fit into an `int`).

Time Plan and Checkpoints

The lab has **10 steps**. Each step ends with a **checkpoint**: you submit your work for that step before moving on. The "finish by" column is the minute (from the start of the lab) by which you should have submitted.

Checkpoint	Topic	Time	Finish by
Setup	Create the file, paste the starter code	5 min	min 5
Step 1	Classes and objects	5 min	min 10
Step 2	Constructors and this. shorthand	7 min	min 17
Step 3	Named constructors	7 min	min 24
Step 4	Factory constructors	8 min	min 32
Step 5	Initializer lists and assertions	9 min	min 41
Step 6	Getters	6 min	min 47
Step 7	Setters	7 min	min 54
Step 8	A menu built from text (toString, collections)	7 min	min 61
Step 9	Building a receipt	7 min	min 68
Step 10	Capstone: discount coupons	12 min	min 80
Wrap-up	Format, analyse, reflection questions	10 min	min 90
Total		90 min	

What to submit at every checkpoint

Submit three things, in this order (a message, form or LMS box, as your instructor tells you):

1. Your **seed line**: the first line your program prints, e.g. `Seed: 47 (t=4, u=7)`
2. The **code** you wrote or changed in this step (just the new or changed parts, as text, not a screenshot)
3. The **output** your program printed for this step (copy the lines that start with `Step N:`)

```
Roll no: 21F-0347          Step: 3
--- CODE ---
(paste the code you wrote in this step)
--- OUTPUT ---
(paste the lines printed by this step)
```

Your answers are checked against an answer key generated for your own roll number. Output that is correct for a different roll number will not match, so work from your own seed.

Setup (5 min)

1. Create a folder `lab3` and a file `lab3.dart` (or open dartpad.dev).
2. Paste the starter code below. Replace the roll number text with **your** roll number, which must end in two digits.

- Run it with `dart run --enable-asserts lab3.dart`. The `--enable-asserts` flag matters from Step 5 onward. (DartPad has asserts switched on already.)
- You should see your seed line and ten step headings.

```
// lab3.dart - Campus Cafe Order System
// Name: _____ Roll no: _____

const String rollNo = 'YOUR-ROLL-NO-HERE'; // e.g. '2100672347'

// ===== Seeded settings (generated from YOUR roll number). Do not edit. =====
final int seed = int.parse(rollNo.substring(rollNo.length - 2));
final int t = seed ~/ 10; // tens digit
final int u = seed % 10; // units digit

const List<String> menu = [
  'Chai', 'Latte', 'Mocha', 'Samosa', 'Brownie',
  'Sandwich', 'Cold Coffee', 'Fries', 'Pakora', 'Zinger Wrap',
];
int priceOf(int i) => 100 + 7 * i + 3 * t; // price of menu[i], in rupees
final int priceFloor = 60 + 5 * t;
final int taxPercent = 5 + t;
final int bigOrderLimit = 450 + 20 * t;
final int balanceCap = 600 + 20 * t;
final int couponPercent = 5 + t + u;
// =====

void main() {
  print('Seed: $seed (t=$t, u=$u)');
  step1();
  step2();
  step3();
  step4();
  step5();
  step6();
  step7();
  step8();
  step9();
  step10();
}

void step1() { print('--- Step 1 ---'); }
void step2() { print('--- Step 2 ---'); }
void step3() { print('--- Step 3 ---'); }
void step4() { print('--- Step 4 ---'); }
void step5() { print('--- Step 5 ---'); }
void step6() { print('--- Step 6 ---'); }
void step7() { print('--- Step 7 ---'); }
void step8() { print('--- Step 8 ---'); }
void step9() { print('--- Step 9 ---'); }
void step10() { print('--- Step 10 ---'); }
```

If you get a `FormatException` straight away, you have not replaced the placeholder roll number, or your roll number does not end in two digits. Put the classes you write above `main()`, and your step code inside the matching `stepN()` function.

Step 1: Classes and Objects (5 min)

Topic: Classes & Objects Basics. A class defines fields; every object gets its own independent copy.

Task 1.1

Write `class Dish` with two fields: `late String name;` and `late int price;`. (`late` promises Dart you will assign the field before you read it. A plain non-nullable field with no value does not compile in null-safe Dart, so the slide's `Cat` example needs `late` or a `?` type in a real program.)

Task 1.2

In `step1()` create two objects, with no `new` keyword:

- `item1`: name is `menu[u]`, price is `priceOf(u)`.
- `item2`: use index `(u + 1) % 10` for both the name and the price. **After** creating it, give a discount of `u` rupees: `item2.price = item2.price - u;`

Print both items **after** the discount is applied:

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 1: <item1 name> Rs <item1 price>
Step 1: <item2 name> Rs <item2 price>
```

Self-check: `item1.price` is between 100 and 190. Changing `item2` must not change `item1`.

SUBMIT. Checkpoint 1: seed line, the `Dish` class and `step1()` code, the 2 printed lines.

Step 2: Constructors and the `this`. Shorthand (7 min)

Topic: Constructors & this. Shorthand. A constructor runs on creation and can contain logic.

Task 2.1

Write `class MenuItem` with fields `String name;` and `int price;`. First write the **verbose** constructor, `MenuItem(String name, int price) { this.name = name; this.price = price; }`, run it, and then replace it with the shorthand `MenuItem(this.name, this.price)`. Both give identical results.

Task 2.2

Add logic to the constructor **body**: if `this.price` is below `priceFloor`, raise it to `priceFloor`. Use `this.price` inside the body.

Task 2.3

In `step2()` create `a` using `MenuItem(menu[u], priceOf(u))` and `b` using `MenuItem('Test Special', 15 * u)`. Print:

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 2: <a.name> Rs <a.price>
Step 2: Test Special Rs <b.price>
```

Self-check: `b.price` is never below your `priceFloor` (60 to 105).

Think (answer in a code comment): Why could `price` not be declared `final` in this version of the class?

SUBMIT. Checkpoint 2: seed line, the `MenuItem` class and `step2()` code, the 2 printed lines.

Step 3: Named Constructors (7 min)

Topic: Named Constructors. `ClassName.name()` gives a class several ways to build an object.

Task 3.1

Add a named constructor `MenuItem.free(this.name) : price = 0;` for giveaway items.

Task 3.2

Add `MenuItem.fromString(String text)` that reads text like `'Chai:135'` and builds the item. Use an **initializer list** (after the `:`) with `text.split(':')` and `int.parse(...)`.

Task 3.3

In `step3()` create `freebie = MenuItem.free('Water')`, and `parsed` from the string `'${menu[i]}:${priceOf(i)}'` where `i = (u + 2) % 10`. Print:

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 3: <freebie.name> Rs <freebie.price>
Step 3: <parsed.name> Rs <parsed.price>
Step 3: floor=<priceFloor>, free price=<freebie.price>
```

Think (answer in a code comment): The floor is, say, 80 but `free()` produced 0. Why did the floor logic not run?

SUBMIT. Checkpoint 3: seed line, the two new constructors and `step3()` code, the 3 printed lines.

Step 4: Factory Constructors (8 min)

Topic: Factory Constructors. A factory may return an existing instance instead of a new one.

Task 4.1

Complete this singleton, which keeps one shared log for the whole cafe. Replace the `TODO` with a `factory OrderLog()` that returns the one instance, creating it only the first time (`_instance ??= ...`).

```
class OrderLog {
    static OrderLog? _instance;
    final List<String> entries = [];

    OrderLog._internal();                // private named constructor

    // TODO: factory OrderLog() { ... }    // always the same object

    void add(String msg) => entries.add(msg);
}
```

Task 4.2

In `step4()` create `log1 = OrderLog()` and `log2 = OrderLog()`. Then loop `i` from 1 to `u + 2` (inclusive). For each `i`, build the message `'order ${100 * t + i}'`; add it through `log1` when `i` is odd and through `log2` when `i` is even. Print:

PRINT IN THIS FORMAT (replace the <...> parts with your own values)

```
Step 4: same object? <true or false>
Step 4: entries = <log1.entries.length>
Step 4: last = <log2.entries.last>
```

Self-check: `entries` equals `u + 2`, so it is between 2 and 11, even though you used two variables.

Think (answer in a code comment): Why do `_instance` and `_internal` start with an underscore? What could go wrong if they did not?

SUBMIT. Checkpoint 4: seed line, the `OrderLog` class and `step4()` code, the 3 printed lines.

Step 5:_INITIALIZER Lists and Assertions (9 min)

Topic:_INITIALIZER Lists & Assertions. The part after the colon runs before the constructor body.

Task 5.1

Write `class OrderLine` with four `final` fields: `MenuItem item`, `int qty`, `int total`, `int tax`. The constructor takes only `item` and `qty` (`OrderLine(this.item, this.qty)`) and uses an **initializer list** to compute the other two:

- `total` is `item.price * qty`
- `tax` is the line total times `taxPercent`, divided by 100 using integer division (`~/`)
- an `assert` that `qty > 0`, with the message `'qty must be positive'`

```
// the shape of the constructor (fill in the ... parts)
OrderLine(this.item, this.qty)
  : total = ...,
    tax = ...,
    assert(..., '...');
```

Task 5.2

Write a top-level function `OrderLine mainOrder()` that returns `OrderLine(MenuItem(menu[u], priceOf(u)), 2 + (t + u) % 5)`. You will reuse it in later steps.

Task 5.3

In `step5()` call `mainOrder()` and print the first two lines below. Then use this block to prove that the assert works:

```
try {
  OrderLine(line.item, 0);           // line = your mainOrder() result
  print('Step 5: assert did NOT fire');
} on AssertionError {
  print('Step 5: assert fired');
}
```

PRINT IN THIS FORMAT (replace the <...> parts with your own values)

```
Step 5: <item name> x<qty>
Step 5: total=<total> tax=<tax>
Step 5: assert fired
```

Self-check: `qty` is between 2 and 6. If you see "did NOT fire", run with `dart run --enable-asserts lab3.dart`: assertions are a debug-only feature.

Think (answer in a code comment): Try writing `tax = total * taxPercent ~/ 100`. Read the error. Why can an initializer list not read another field of the same object?

SUBMIT. Checkpoint 5: seed line, the `OrderLine` class, `mainOrder()` and `step5()` code, the 3 printed lines.

Step 6: Getters (6 min)

Topic: Getters and Setters (getters). A getter exposes computed data and is used without parentheses.

Task 6.1

Add three getters inside `OrderLine`:

- `int get grand` returns `total + tax`
- `bool get isBigOrder` returns `true` when `grand` is greater than `bigOrderLimit`
- `String get label` returns the item name, then `x`, then the quantity (for example `Chai x3`)

Task 6.2

In `step6()` use `mainOrder()` and print (note: call getters **without** `()`):

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 6: grand=<grand>
Step 6: big order? <true or false> (limit <bigOrderLimit>)
Step 6: label=<label>
```

Task 6.3

Temporarily add `line.grand = 5;`, read the compile error, then delete the line.

Think (answer in a code comment): Why does that line fail? What would you have to add to make it legal?

SUBMIT. Checkpoint 6: seed line, the three getters and `step6()` code, the 3 printed lines.

Step 7: Setters (7 min)

Topic: Getters and Setters (setters). A setter intercepts assignment so you can validate it.

Task 7.1

Finish this class. The setter must keep the balance between 0 and `balanceCap`: a value below 0 becomes 0, a value above `balanceCap` becomes `balanceCap`, anything else is stored as is.

```
class StudentCard {
    final String owner;
    int _balance;                                // private backing field

    StudentCard(this.owner) : _balance = 0;

    int get balance => _balance;

    // TODO: set balance(int v) { ... }
}
```

Heads-up: the slide's setter example (`set width(...) { width = ... }`) will not compile, because a setter cannot share its name with a field that stores the data. Keep the data in a private `_balance` as above. Also, Dart's privacy works per **file**, so inside `lab3.dart` you could write `card._balance = 5` and skip validation. Do not.

Task 7.2

In `step7()` create `card = StudentCard('$seed')` and run these four assignments, printing after each one:

```
card.balance = seed * 10 + 50;
card.balance = -seed - 1;
card.balance = balanceCap - u;
card.balance = card.balance - mainOrder().grand; // pay the main order
```

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 7: topped up -> <card.balance>
Step 7: bad value -> <card.balance>
Step 7: reset -> <card.balance>
Step 7: paid order -> <card.balance>
```

Self-check: no value is ever below 0 or above your `balanceCap` (600 to 780).

Think (answer in a code comment): The setter silently clamps a bad value. What is one other thing a setter could do with an invalid value?

SUBMIT. Checkpoint 7: seed line, the `StudentCard` class and `step7()` code, the 4 printed lines.

Step 8: A Menu Built from Text (7 min)

Topics: named constructors, `toString()`, and collection methods from Session 2 working together.

Task 8.1

Add `@override String toString() => '$name (Rs $price)';` to `MenuItem`, so a menu item prints nicely instead of as `Instance of 'MenuItem'`.

Task 8.2

Write a top-level `List<MenuItem> buildMenu()` that builds 4 items. For `k` from 0 to 3, let `idx = (u + 3 * k) % 10`, make the text `'${menu[idx]}:${priceOf(idx)}'`, and turn it into an item with `MenuItem.fromString`. A collection-for (`[for (...) ...]`) keeps this short.

Task 8.3

In `step8()` call `buildMenu()`, find the most expensive item with `reduce`, and add up all four prices with `fold`. Print:

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 8: menu = <the whole list>
Step 8: priciest = <name of the most expensive item>
Step 8: sum = <total of the 4 prices>
```

SUBMIT. Checkpoint 8: seed line, `toString()`, `buildMenu()` and `step8()` code, the 3 printed lines.

Step 9: Building a Receipt (7 min)

Topics: reuse of classes, getters, and the shared log from Step 4.

Task 9.1

Write a top-level `List<OrderLine> buildReceipt()`. Take the **first three** items from `buildMenu()`. For `k = 0, 1, 2`, make `OrderLine(items[k], 1 + (t + k) % 4)`.

Task 9.2

In `step9()` go through the receipt lines. For each line print its label and `grand`, and also add the text `'receipt: ${line.label}'` to the shared log with `OrderLog().add(...)`. After the loop print the sum of all `grand` values and the current size of the log:

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 9: <label> = <grand>          (three lines, one per receipt line)
Step 9: receipt total = <sum of the three grand values>
Step 9: log size = <OrderLog().entries.length>
```

Self-check: the log size equals `u + 5`: your Step 4 entries are still there, because `OrderLog()` always returns the same object.

SUBMIT. Checkpoint 9: seed line, `buildReceipt()` and `step9()` code, the 5 printed lines.

Step 10: Capstone, Discount Coupons (12 min)

Topics: everything so far in one class: initializer list, assert, factory with a cache, and a method.

Task 10.1

Write `class Coupon` with `final String code`, `final int percent` and `final int minSpend`. Use this shape:

```
class Coupon {
    static final Map<String, Coupon> _cache = {};

    final String code;
    final int percent;
    final int minSpend;

    // main constructor Coupon(this.code, this.percent):
    //   initializer list: minSpend = percent * 70
    //   assert that percent is between 1 and 50 (inclusive)

    // factory Coupon.fromCode(String code): return the coupon stored in
    // _cache for this code; if there is none, create Coupon(code, couponPercent),
    // store it, and return it (hint: _cache.putIfAbsent)

    // int discountOn(int amount): if amount >= minSpend return
    //   amount * percent ~/ 100, otherwise return 0
}
```

Task 10.2

In `step10()` do the following:

1. Build the code text `'CAFE${seed.toString().padLeft(2, '0')}`' (so seed 7 becomes `CAFE07`).
2. Create `c1` and `c2`, both with `Coupon.fromCode(code)`.
3. Add up `grand` over all lines of `buildReceipt()` to get the receipt amount.
4. Compute the discount with `c1.discountOn(receipt)` and print:

PRINT IN THIS FORMAT (replace the `<...>` parts with your own values)

```
Step 10: <code> gives <percent>% off, min spend <minSpend>
Step 10: cached? <true or false>
Step 10: receipt <receipt>, discount <discount>, payable <receipt - discount>
```

Self-check: `cached?` must be `true`. If your receipt is below `minSpend` the discount is 0, which is a correct result for some roll numbers.

SUBMIT. Checkpoint 10: seed line, the `Coupon` class and `step10()` code, the 3 printed lines, and your complete `lab3.dart` file.

Wrap-up (10 min)

1. Run `dart format .` and `dart analyze` (or DartPad's Format button). Fix every warning.
2. Run the whole program once from top to bottom and confirm that all ten steps print.
3. Answer these four questions as comments at the bottom of `lab3.dart` (one or two sentences each), and include them in your final submission:

Q1. `Animal(this.name, this.type);` and the verbose constructor give the same result. What does the shorthand save you?

Q2. When would you choose a **named** constructor, and when a **factory** constructor?

Q3. What is the difference between assigning a field in a constructor **body** and assigning it in an **initializer list**?

Q4. Give one reason to use a getter instead of storing the value in a field, and one reason to use a setter instead of a public field.

Common Pitfalls

- A non-nullable field with no value and no constructor does not compile. Use `late`, a nullable type, or initialise it in a constructor.
- Forgetting `~/`: `price * qty * taxPercent / 100` is a `double` and will not fit into an `int` field.
- A setter cannot have the same name as the field that stores its value. Use `_balance` with `balance`.
- An initializer list cannot read other fields of the same object. Recompute from the constructor parameters.
- `assert` is ignored unless asserts are enabled (`--enable-asserts`, or DartPad).
- Calling a getter with parentheses (`line.grand()`) is an error. Getters are used like fields.

Marking Scheme

Section	Marks	What is checked
Steps 1 to 10 (8 marks each)	80	Output matches the answer key for the student's own seed (4); code uses the required feature correctly (3); submitted at its checkpoint (1)
Final file	10	Complete lab3.dart runs, is formatted, and has no analyzer warnings
Reflection	10	Answers to Q1 to Q4 and the Think comments
Total	100	